

Documento 1: Errores de Configuración Build/Gradle
SponsorFlowNexus - Modulo WhatsApp
Fecha: 4 de Marzo 2026

Resumen Ejecutivo

Se identificaron 20 errores en la configuración de build del proyecto SponsorFlow Nexus. Los errores CRITICOS son: (1) Version del Compose Compiler 1.5.8 incompatible con Kotlin 1.9.22 que requiere 1.5.10, causando fallo de compilación inmediato; (2) Version incorrecta de androidx.test.ext:junit usando '1.5.0' que no existe para ese artefacto (la correcta es '1.1.5'); (3) Version de Gradle en el wrapper no verificable pero critica para compatibilidad con AGP 8.2.2 que requiere minimo Gradle 8.2. Los errores de severidad ALTA incluyen: propiedad org.gradle.jvmargs duplicada en gradle.properties, plugin KSP sin version centralizada en [versions], AGP sin version centralizada y falta del plugin android-library en el catalogo, bloque plugins duplicado en settings.gradle que conflictua con el Version Catalog, isMinifyEnabled=false en release build con datos sensibles, falta de testInstrumentationRunner en defaultConfig que impide ejecutar tests instrumentados, y riesgo de mezcla KAPT/KSP. Los errores MEDIOS y BAJOS incluyen: dependencia sqllite-ktx hardcodeada fuera del catalogo, uso de version alpha de security-crypto en produccion, version de Compose BOM no actualizada, colision de alias 'hilt-android' entre plugins y libraries, incompatibilidad potencial entre OkHttp 4.12.0 y Retrofit 2.9.0, y problemas de arquitectura hibrida con viewBinding + Compose activos simultaneamente.

Total de errores: 20

ERR-001 - Propiedad 'org.gradle.jvmargs' duplicada en gradle.properties

Archivo: gradle.properties

Ubicación: Línea 3 y Línea 19

Severidad: HIGH

Código actual (con error):

...

```
org.gradle.jvmargs=-Xmx2048m -Dfile.encoding=UTF-8
```

...

```
org.gradle.jvmargs=-Xmx4096m -XX:+HeapDumpOnOutOfMemoryError  
-Dfile.encoding=UTF-8
```

...

Código corregido:

...

```
org.gradle.jvmargs=-Xmx4096m -XX:+HeapDumpOnOutOfMemoryError  
-Dfile.encoding=UTF-8
```

...

Explicación: Gradle procesa las propiedades de forma secuencial y la segunda definición sobrescribe la primera de manera silenciosa e impredecible según la implementación del JVM y la versión de Gradle. Esto puede causar comportamientos inconsistentes entre

entornos locales y CI. Se debe mantener unicamente la segunda definicion mas completa (4096m con HeapDumpOnOutOfMemoryError) y eliminar la primera linea duplicada.

ERR-002 - Version del Compose Compiler incompatible con Kotlin 1.9.22

Archivo: libs.versions.toml

Ubicacion: [versions] composeCompiler = "1.5.8"

Severidad: CRITICAL

Codigo actual (con error):

...

composeCompiler = "1.5.8"

...

Codigo corregido:

...

composeCompiler = "1.5.10"

...

Explicacion: La version de Compose Compiler 1.5.8 corresponde a Kotlin 1.9.20. Para Kotlin 1.9.22 la version correcta del compilador de Compose es 1.5.10 segun la tabla oficial de compatibilidad de AndroidX

(<https://developer.android.com/jetpack/androidx/releases/compose-kotlin>). Usar una version desalineada produce error de compilacion: 'This version (1.5.8) of the Compose Compiler requires Kotlin version 1.9.20 but 1.9.22 was found'.

ERR-003 - Plugin KSP en [plugins] usa version literal en lugar de version.ref apuntando a una version centralizada, y la version de KSP no esta definida en [versions]

Archivo: libs.versions.toml

Ubicacion: [plugins] ksp = { id = "com.google.devtools.ksp", version = "1.9.22-1.0.17" }

Severidad: HIGH

Codigo actual (con error):

...

ksp = { id = "com.google.devtools.ksp", version = "1.9.22-1.0.17" }

...

Codigo corregido:

...

ksp = { id = "com.google.devtools.ksp", version.ref = "ksp" }

Y agregar en [versions]:

ksp = "1.9.22-1.0.17"

...

Explicacion: La version 1.9.22-1.0.17 del plugin KSP esta hardcodeda directamente en la seccion [plugins] del TOML, pero la misma version aparece tambien hardcodeda en root build.gradle y settings.gradle. Si se actualiza en un lugar no se actualiza en los otros, generando inconsistencias. Ademas, por convencion y mantenibilidad, todas las versiones deben estar en [versions]. Es critico que la version KSP coincida EXACTAMENTE con la version de Kotlin (1.9.22): la parte '1.9.22' del artefacto '1.9.22-1.0.17' debe coincidir con kotlin = '1.9.22'.

ERR-004 - Plugin AGP (Android Gradle Plugin) sin version centralizada en [versions] y falta plugin android-library en el catalogo

Archivo: libs.versions.toml

Ubicacion: [plugins] android-application = { id = "com.android.application", version = "8.2.2" }

Severidad: HIGH

Codigo actual (con error):

...

```
android-application = { id = "com.android.application", version = "8.2.2" }
```

...

Codigo corregido:

...

```
android-library = { id = "com.android.library", version.ref = "agp" }
```

Y agregar en [versions]:

```
agp = "8.2.2"
```

Modificar plugin existente:

```
android-application = { id = "com.android.application", version.ref = "agp" }
```

...

Explicacion: La version de AGP '8.2.2' esta hardcodeda directamente en la declaracion del plugin en [plugins] sin referenciar [versions]. Ademas, aunque en root build.gradle y settings.gradle se declara 'com.android.library', este plugin NO existe en la seccion [plugins] del TOML. Si algun modulo de tipo library necesita alias(libs.plugins.android.library) fallara con 'Plugin not found'. Se debe centralizar la version AGP en [versions] y agregar la entrada faltante.

ERR-005 - Bloque plugins{} dentro de pluginManagement en settings.gradle duplica y puede generar conflictos con las versiones declaradas en root build.gradle y el Version Catalog

Archivo: settings.gradle.kts

Ubicacion: pluginManagement { plugins { ... } }

Severidad: HIGH

Codigo actual (con error):

```

...
pluginManagement {
    repositories {
        google()
        mavenCentral()
        gradlePluginPortal()
    }

    plugins {
        id("com.android.application") version "8.2.2"
        id("com.android.library") version "8.2.2"
        id("org.jetbrains.kotlin.android") version "1.9.22"
        id("com.google.devtools.ksp") version "1.9.22-1.0.17"
        id("com.google.dagger.hilt.android") version "2.50"
        id("org.jetbrains.kotlin.plugin.serialization") version "1.9.22"
    }
}
...

```

Codigo corregido:

```

...

```

ELIMINAR el bloque plugins{} de settings.gradle por completo.

La resolucion de plugins ya esta gestionada por el Version Catalog (libs.versions.toml)

y por el bloque plugins{} en root build.gradle.

settings.gradle debe quedar:

```

pluginManagement {
    repositories {
        google()
        mavenCentral()
        gradlePluginPortal()
    }
}

dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
    repositories {
        google()
        mavenCentral()
        maven { url = uri("https://jitpack.io") }
    }
}

rootProject.name = "SponsorFlowNexus"
include(":app")
...

```

Explicacion: Cuando se usa un Version Catalog (libs.versions.toml) junto con el bloque `plugins{}` en root `build.gradle`, definir nuevamente los plugins con versiones explicitas en el bloque `pluginManagement { plugins {} }` de `settings.gradle` crea una triple declaracion de versiones. Esto puede causar: (1) conflictos de resolucion de version donde Gradle no sabe cual priorizar, (2) el bloque `plugins{}` en `pluginManagement` de `settings.gradle` tiene menor prioridad que las versiones del catalogo pero genera confusion y warnings. La practica correcta con Version Catalogs es NO redeclarar versiones de plugins en `settings.gradle`.

ERR-006 - `isMinifyEnabled = false` en release build con Hilt, Room y Retrofit sin reglas de ProGuard activas

Archivo: `app/build.gradle.kts`

Ubicacion: `buildTypes { release { isMinifyEnabled = false } }`

Severidad: HIGH

Codigo actual (con error):

```
...
release {
    isMinifyEnabled = false
    proguardFiles(
        getDefaultProguardFile("proguard-android-optimize.txt"),
        "proguard-rules.pro"
    )
}
...
```

Codigo corregido:

```
...
release {
    isMinifyEnabled = true
    isShrinkResources = true
    proguardFiles(
        getDefaultProguardFile("proguard-android-optimize.txt"),
        "proguard-rules.pro"
    )
}
...
```

Explicacion: Con `isMinifyEnabled = false` la app de produccion no tiene ofuscacion ni minificacion. Esto es un problema critico de seguridad en una app que usa autenticacion (play-services-auth, play-integrity) y maneja URLs de servidor en `BuildConfig`. Ademias, las reglas de ProGuard configuradas en `proguardFiles()` son completamente ignoradas cuando `isMinifyEnabled` es false, haciendo que la configuracion de ProGuard sea letra muerta. Hilt requiere que las reglas de ProGuard esten activas para funcionar correctamente en release. Se debe habilitar minificacion y agregar `isShrinkResources = true`.

ERR-007 - Dependencia androidx.sqlite.sqlite-ktx declarada con string literal en lugar de usar el Version Catalog

Archivo: app/build.gradle.kts

Ubicacion: implementation("androidx.sqlite.sqlite-ktx:2.4.0")

Severidad: MEDIUM

Codigo actual (con error):

```
...  
implementation("androidx.sqlite.sqlite-ktx:2.4.0")  
...
```

Codigo corregido:

```
...  
Agregar en libs.versions.toml [versions]:  
sqliteKtx = "2.4.0"
```

```
Agregar en libs.versions.toml [libraries]:  
androidx-sqlite-ktx = { module = "androidx.sqlite.sqlite-ktx", version.ref = "sqliteKtx" }
```

```
Cambiar en app/build.gradle.kts:  
implementation(libs.androidx.sqlite.ktx)  
...
```

Explicacion: La dependencia 'androidx.sqlite.sqlite-ktx:2.4.0' esta hardcodeada directamente en app/build.gradle.kts como string literal, violando el patron del proyecto que usa Version Catalog (libs.versions.toml) para todas las demas dependencias. Esto impide la gestion centralizada de versiones y puede llevar a inconsistencias. Ademias, Room 2.6.1 ya incluye una dependencia transitiva sobre sqlite-ktx 2.4.0, por lo que esta declaracion explicita podria ser completamente innecesaria. Si se mantiene, debe moverse al catalogo.

ERR-008 - androidx.security:security-crypto version alpha en uso para produccion

Archivo: libs.versions.toml

Ubicacion: [versions] security = "1.1.0-alpha06"

Severidad: MEDIUM

Codigo actual (con error):

```
...  
security = "1.1.0-alpha06"  
...
```

Codigo corregido:

```
...  
En libs.versions.toml [versions], cambiar:  
security = "1.0.0"
```

O si se requiere la version alpha, documentar explicitamente:
security = "1.1.0-alpha06" # NOTA: version alpha, inestable para produccion

Explicacion: La version 1.1.0-alpha06 de androidx.security:security-crypto es una version alpha inestable. La version estable actual es 1.0.0. Usar versiones alpha en produccion puede introducir bugs, cambios de API sin notice, y la Play Store puede rechazar apps con dependencias alpha en ciertos contextos de revision. La version estable 1.0.0 es completamente funcional para EncryptedSharedPreferences y EncryptedFile. Si se necesitan features especificos de 1.1.0 se debe documentar y aceptar el riesgo explicitamente.

ERR-009 - Version incorrecta para androidx.test.ext:junit - usa version de androidx.test:runner en lugar de la version correcta del artefacto junit

Archivo: libs.versions.toml

Ubicacion: [versions] androidxTest = "1.5.0"

Severidad: CRITICAL

Codigo actual (con error):

```
androidxTest = "1.5.0"
```

```
androidx-test-ext-junit = { module = "androidx.test.ext:junit", version.ref = "androidxTest" }
```

Codigo corregido:

Agregar en libs.versions.toml [versions]:

```
androidxTestExtJunit = "1.1.5"
```

Cambiar en [versions]:

```
androidxTest = "1.1.5"
```

La entrada en [libraries] ya esta correcta en modulo, solo la version es incorrecta:

```
androidx-test-ext-junit = { module = "androidx.test.ext:junit", version.ref =  
"androidxTestExtJunit" }
```

Explicacion: La version '1.5.0' corresponde al artefacto 'androidx.test:runner', NO a 'androidx.test.ext:junit'. La version correcta y estable de 'androidx.test.ext:junit' es '1.1.5'. Usar la version incorrecta causara un error de resolucion de dependencia: 'Could not find androidx.test.ext:junit:1.5.0'. Estos son artefactos distintos con ciclos de version independientes.

ERR-010 - Uso de material-icons-extended que agrega mas de 17MB al APK sin necesidad aparente

Archivo: libs.versions.toml / app/build.gradle.kts

Ubicacion: [libraries] androidx-compose-material-icons / dependencies {
implementation(libs.androidx.compose.material.icons) }

Severidad: MEDIUM

Codigo actual (con error):

```
...  
  
androidx-compose-material-icons = { module =  
"androidx.compose.material:material-icons-extended" }  
  
implementation(libs.androidx.compose.material.icons)  
...
```

Codigo corregido:

```
...  
  
El alias correcto en Kotlin DSL para guiones en nombres es con punto:  
libs.androidx.compose.material.icons (correcto para material-icons-extended)  
Verificar que el alias generado por el TOML para:  
aandroidx-compose-material-icons = { module =  
"androidx.compose.material:material-icons-extended" }  
Se accede en build.gradle.kts como:  
implementation(libs.androidx.compose.material.icons)  
ADVERTENCIA: material-icons-extended es un artefacto extremadamente grande (+17MB).  
Considerar usar en su lugar:  
implementation(libs.androidx.compose.material.icons.core)  
Y agregar en TOML:  
aandroidx-compose-material-icons-core = { module =  
"androidx.compose.material:material-icons-core" }  
...
```

Explicacion: El artefacto 'androidx.compose.material:material-icons-extended' contiene TODOS los iconos de Material Design (mas de 900) y agrega aproximadamente 17-25MB al APK final. Dado que isMinifyEnabled=false en release (ver ERR-006), estos iconos no seran eliminados por tree-shaking. La practica recomendada por Google es usar 'material-icons-core' (que incluye los iconos mas comunes) y solo agregar extended si se necesitan iconos especificos. Si se habilita minificacion y R8, los iconos no usados seran eliminados, pero la compilacion sera mas lenta.

ERR-011 - Version de Compose BOM 2024.02.00 tiene vulnerabilidades conocidas y no es la ultima de su serie

Archivo: libs.versions.toml

Ubicacion: [versions] compose = "2024.02.00"

Severidad: MEDIUM

Codigo actual (con error):

...

```
compose = "2024.02.00"
```

...

Codigo corregido:

...

Agregar en libs.versions.toml [versions]:

```
compose = "2024.02.02"
```

O la version LTS disponible:

```
compose = "2024.04.01"
```

...

Explicacion: La version 2024.02.00 del BOM de Compose fue la primera release de febrero 2024. Google publico rapidamente 2024.02.02 con correcciones importantes de bugs y la version 2024.04.01 es la LTS recomendada. La version 2024.02.00 tiene issues conocidos con animaciones en API 34 y con el compilador 1.5.10. Usar la version mas actualizada de la serie o la LTS garantiza compatibilidad y estabilidad.

ERR-012 - Colision de alias 'hilt-android' entre seccion [plugins] y [libraries] en el Version Catalog

Archivo: libs.versions.toml

Ubicacion: [plugins] hilt-android y [libraries] hilt-android

Severidad: MEDIUM

Codigo actual (con error):

...

En [plugins]:

```
hilt-android = { id = "com.google.dagger.hilt.android", version.ref = "hilt" }
```

En [libraries]:

```
hilt-android = { module = "com.google.dagger:hilt-android", version.ref = "hilt" }
```

...

Codigo corregido:

...

Verificar que en el TOML el alias 'hilt-android' en [plugins] no colisione con el alias 'hilt-android' en [libraries].

En [plugins] esta definido como:

```
hilt-android = { id = "com.google.dagger.hilt.android", version.ref = "hilt" }
```

En [libraries] esta definido como:

```
hilt-android = { module = "com.google.dagger:hilt-android", version.ref = "hilt" }
```

Esto genera ambigüedad. Renombrar el plugin en [plugins]:

```
hilt-gradle-plugin = { id = "com.google.dagger.hilt.android", version.ref = "hilt" }
```

Y actualizar en app/build.gradle.kts:
alias(libs.plugins.hilt.gradle.plugin)

Y en root build.gradle:
id("com.google.dagger.hilt.android") version "2.50" apply false

Explicacion: El Version Catalog de Gradle genera accessors para tanto libraries como plugins usando el mismo mecanismo de nomenclatura basado en los alias. Tener 'hilt-android' definido tanto en [plugins] como en [libraries] puede causar confusion en el IDE y potencialmente errores de resolucion de accessor en Kotlin DSL, ya que libs.plugins.hilt.android y libs.hilt.android podrian conflictuar en el modelo de objetos generado. Aunque tecnicamente estan en namespaces separados (libs.plugins.* vs libs.*), es una practica que genera confusion y debe evitarse con nombres descriptivos unicos.

ERR-013 - Falta androidx.activity:activity-ktx en el catalogo cuando se usa navegacion con fragmentos

Archivo: libs.versions.toml

Ubicacion: [libraries] - entrada faltante

Severidad: LOW

Codigo actual (con error):

No existe androidx-activity-ktx en el catalogo

Solo existe:

```
androidx-activity-compose = { module = "androidx.activity:activity-compose", version.ref = "activity" }
```

Codigo corregido:

Agregar en [versions]:
activityKtx = "1.8.2"

Agregar en [libraries]:
androidx-activity-ktx = { module = "androidx.activity:activity-ktx", version.ref = "activityKtx" }

NOTA: activity-compose ya incluye activity-ktx como dependencia transitiva, pero para Navigation con fragmentos se recomienda declararlo explicitamente.

Explicacion: El proyecto usa tanto navigation-fragment-ktx como navigation-compose simultaneamente, lo que indica una arquitectura mixta (Fragments + Compose). La libreria activity-ktx es necesaria para usar by viewModels(), activityViewModels(), y el manejo correcto del ciclo de vida en actividades con soporte Kotlin. Aunque puede llegar

transitivamente via otros artefactos, no declararla explicitamente es una dependencia implicita fragil que puede romperse si las dependencias transitivas cambian.

ERR-014 - Falta testInstrumentationRunner en defaultConfig y falta dependencia

androidx.test:runner

Archivo: app/build.gradle.kts

Ubicacion: defaultConfig { } - propiedad faltante

Severidad: HIGH

Codigo actual (con error):

...

```
defaultConfig {  
    applicationId = "com.sponsorflow.nexus"  
    minSdk = 26  
    targetSdk = 34  
    versionCode = 24  
    versionName = "2.4.0"  
    // No hay testInstrumentationRunner  
}
```

...

Codigo corregido:

...

Agregar en libs.versions.toml [versions]:

androidxTestRunner = "1.5.2"

Agregar en [libraries]:

```
androidx-test-runner = { module = "androidx.test:runner", version.ref = "androidxTestRunner"  
}
```

Agregar en defaultConfig de app/build.gradle.kts:

```
defaultConfig {  
    ...  
    testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"  
}
```

...

Explicacion: El proyecto tiene dependencias de androidTest (espresso, junit ext) pero no declara 'testInstrumentationRunner' en defaultConfig. Sin esta propiedad, los tests instrumentados no pueden ejecutarse. El valor por defecto de Gradle es 'android.test.InstrumentationTestRunner' que es obsoleto y no compatible con las APIs modernas de AndroidX Test. Se debe declarar explicitamente 'androidx.test.runner.AndroidJUnitRunner' y agregar la dependencia runner correspondiente.

ERR-015 - Archivo gradle-wrapper.properties no proporcionado - version de Gradle no verificable para compatibilidad con AGP 8.2.2

Archivo: gradle/wrapper/gradle-wrapper.properties

Ubicacion: distributionUrl

Severidad: CRITICAL

Codigo actual (con error):

...

No se proporciono gradle-wrapper.properties pero es critico verificar:
distributionUrl en gradle-wrapper.properties

...

Codigo corregido:

...

La configuracion es compatible pero se debe verificar:

AGP 8.2.2 requiere Gradle 8.2 o superior

Kotlin 1.9.22 es compatible con AGP 8.2.2

Java 17 es compatible y requerido por AGP 8.1+

En gradle/wrapper/gradle-wrapper.properties debe estar:

distributionUrl=https\://services.gradle.org/distributions/gradle-8.6-bin.zip

AGP 8.2.2 es compatible con Gradle 8.2-8.6

Si se usa Gradle < 8.2 con AGP 8.2.2, el build falla con:

'Minimum supported Gradle version is 8.2'

...

Explicacion: AGP 8.2.2 requiere Gradle minimo 8.2. Si el wrapper apunta a una version anterior (como 8.0 o 7.x que son comunes en proyectos no actualizados), el build fallara inmediatamente con 'Minimum supported Gradle version is 8.2. Current version is X.X'. Este es uno de los errores mas comunes al actualizar AGP. La version recomendada para AGP 8.2.2 es Gradle 8.2 a 8.6.

ERR-016 Incompatibilidad de versiones entre OkHttp 4.12.0 y Retrofit 2.9.0

Archivo: libs.versions.toml

Ubicacion: [versions] retrofit = "2.9.0" con okhttp = "4.12.0"

Severidad: MEDIUM

Codigo actual (con error):

...

okhttp = "4.12.0"

retrofit = "2.9.0"

...

Codigo corregido:

...

Agregar en [versions]:
retrofit = "2.11.0"

O mantener 2.9.0 si se requiere compatibilidad especifica, pero documentarlo.
Retrofit 2.9.0 tiene un bug conocido con OkHttp 4.12.0 en la gestion de timeouts. Actualizar a 2.11.0 que es compatible con OkHttp 4.12.0:
retrofit = "2.11.0"
'''

Explicacion: Retrofit 2.9.0 fue publicado en 2022 y tiene dependencia sobre OkHttp 4.x, pero la combinacion especifica con OkHttp 4.12.0 puede presentar problemas de compatibilidad con la API de manejo de respuestas. Retrofit 2.11.0 (2024) esta diseñado y probado con OkHttp 4.12.x. Ademas, Retrofit 2.9.0 no tiene soporte nativo para Kotlin suspend functions en todas las configuraciones, mientras que versiones posteriores mejoran significativamente la integracion con coroutines de Kotlin. Se recomienda actualizar Retrofit a 2.11.0.

ERR-017 - Verificacion de que no haya mezcla de KAPT y KSP - potencial conflicto si hay configuracion heredada
Archivo: app/build.gradle.kts
Ubicacion: ksp dependencies block
Severidad: HIGH

Codigo actual (con error):
'''

```
ksp(libs.hilt.compiler)
ksp(libs.androidx.hilt.compiler)
ksp(libs.androidx.room.compiler)
'''
```

Codigo corregido:
'''

Agregar plugin de Kotlin Compose en root build.gradle (para Kotlin 2.0+)
Con Kotlin 1.9.22 el enfoque actual es correcto usando composeOptions, pero se debe asegurar que el KAPT no este mezclado con KSP.

VERIFICAR que NO exista ninguna referencia a:

```
kapt(libs.hilt.compiler)
kapt(libs.androidx.hilt.compiler)
kapt(libs.androidx.room.compiler)
```

Si existen, deben ser KSP, que ya esta correcto en el build.gradle provisto.

La mezcla de KAPT y KSP en el mismo modulo causa:

```
'KAPT and KSP cannot be used together in the same module for the same annotation processor'
'''
```

Explicacion: El proyecto migro correctamente a KSP para Hilt y Room (lo cual es correcto). Sin embargo, si en el proyecto existe algun archivo de configuracion heredado o si el modulo WhatsApp mencionado en el enunciado usa KAPT para los mismos procesadores, la coexistencia de KAPT y KSP para Hilt/Room en el mismo modulo causa un error de compilacion. Es importante verificar que ningun build.gradle en el proyecto use 'kapt' para dependencias que ya usan 'ksp'.

ERR-018 - Configuration cache habilitado con Hilt 2.50 puede tener compatibilidad limitada

Archivo: gradle.properties

Ubicacion: org.gradle.configuration-cache=true

Severidad: LOW

Codigo actual (con error):

...

org.gradle.configuration-cache=true

...

Codigo corregido:

...

Agregar en gradle.properties:

kotlin.incremental=true

ksp.incremental=true

ksp.incremental.log=false

Si se usa configuration cache con Hilt, asegurar compatibilidad:

Hilt 2.50 tiene soporte experimental para configuration cache.

Agregar si hay problemas:

org.gradle.configuration-cache.problems=warn

...

Explicacion: Hilt con KSP y configuration cache de Gradle tiene soporte experimental en la version 2.50. Si bien Hilt 2.50 mejoró significativamente la compatibilidad con configuration cache, aun puede haber casos edge donde falle con el error: 'Configuration cache entry discarded due to incompatibility'. Esto no impide la compilacion pero puede degradar el rendimiento drasticamente al invalidar el cache constantemente. Se recomienda agregar 'org.gradle.configuration-cache.problems=warn' para diagnosticar sin fallar el build.

ERR-019 - viewBinding y Compose habilitados simultaneamente indica arquitectura mixta no documentada o residuo de migracion

Archivo: app/build.gradle.kts

Ubicacion: buildFeatures { viewBinding = true }

Severidad: LOW

Codigo actual (con error):

```

...
buildFeatures {
    viewBinding = true
    buildConfig = true
    compose = true
}
...

```

Codigo corregido:

Opcion A: Eliminar viewBinding si el proyecto migro completamente a Compose:

```

buildFeatures {
    buildConfig = true
    compose = true
}

```

Opcion B: Mantener viewBinding si se usan ambos (arquitectura mixta):

```

buildFeatures {
    viewBinding = true
    buildConfig = true
    compose = true
}
...

```

Explicacion: Tener viewBinding = true junto con compose = true sugiere que el proyecto usa una arquitectura hibrida con Views tradicionales y Compose. Esto no es un error de compilacion per se, pero incrementa el tiempo de build innecesariamente si el proyecto ya migro completamente a Compose. ViewBinding genera clases para TODOS los layouts XML, incluso los que no se usan. Ademas, si la app usa navigation-fragment-ktx + navigation-compose simultaneamente (como indica el catalogo), se debe documentar explicitamente la estrategia de migracion para evitar confusion y deuda tecnica.

ERR-020 - Verificacion de compatibilidad: hilt-navigation-compose 1.2.0 referenciado via hiltExt pero androidx.hilt:hilt-work tambien usa hiltExt con version 1.2.0

Archivo: libs.versions.toml

Ubicacion: [versions] hiltExt = "1.2.0" compartido entre hilt-work, hilt-compiler y hilt-navigation-compose

Severidad: LOW

Codigo actual (con error):

```

...
androidx-hilt-navigation-compose = { module = "androidx.hilt:hilt-navigation-compose",
version.ref = "hiltExt" }

```

```

hiltExt = "1.2.0"
...

```

Codigo corregido:

'''

Agregar en [bundles] o usar directamente. El bundle 'androidx-compose' en el TOML incluye 'androidx-hilt-navigation-compose' pero esta libreria requiere la version de hiltExt = "1.2.0". Verificar que hilt-navigation-compose 1.2.0 sea compatible con Compose BOM 2024.02.00:

hilt-navigation-compose 1.2.0 requiere:

- Hilt 2.48+ (OK, se usa 2.50)
- Compose Navigation 2.6.0+ (OK, se usa 2.7.7)
- Kotlin 1.8.0+ (OK, se usa 1.9.22)

La compatibilidad es correcta, pero documentarla es importante.

PROBLEMA REAL: androidx.hilt:hilt-navigation-compose version.ref apunta a "hiltExt" pero hiltExt = "1.2.0" - verificar que esta version exista:

La version 1.2.0 de hilt-navigation-compose fue publicada en Feb 2024, es correcta.

'''

Explicacion: El alias 'hiltExt' se reutiliza para tres artefactos distintos: hilt-work, hilt-compiler (androidx) y hilt-navigation-compose. Aunque actualmente todos estan en version 1.2.0, estos artefactos tienen ciclos de release independientes. Si en el futuro se necesita actualizar uno sin el otro, no sera posible con un solo version.ref. Ademas, androidx.hilt:hilt-compiler 1.2.0 es el procesador de anotaciones para hilt-work; si esta version no coincide exactamente con hilt-work, pueden ocurrir errores de generacion de codigo en tiempo de compilacion.

—